

HealthAI v0.1 — AI Healthcare Chatbot

Build the first milestone of a college project called **HealthAI**, an AI-powered healthcare guidance chatbot.

Goal

Create a working web application where a user can send a health-related message through a chat interface, the frontend sends it to a FastAPI backend, the backend sends it to the Gemini API, and the AI response is displayed back in the chat.

For this milestone, **DO NOT implement** authentication, database, ML risk classification, symptom extraction, emergency detection, conversation history, or health profiles yet. Those will be added in later milestones.

1. Technology Stack

Use:

- **Frontend:** HTML, CSS, vanilla JavaScript
- **Backend:** Python + FastAPI
- **AI:** Google Gemini API
- **Server:** Uvicorn
- **Environment variables:** `.env`
- **HTTP communication:** REST API
- **Version control:** Git/GitHub

Keep the architecture modular so additional features can easily be added later.

2. Project Structure

Create the following structure:

```
HealthAI/
|
|— backend/
|   |— main.py
|   |— config.py
|   |— routes/
|   |   |— chat.py
|   |— services/
|   |   |— gemini_service.py
|   |— prompts/
|   |   |— system_prompt.py
|
```

```
├── frontend/
│   ├── index.html
│   ├── style.css
│   └── script.js
├── .env
├── .env.example
├── .gitignore
├── requirements.txt
└── README.md
```

3. Backend Requirements

Create a FastAPI application.

The backend should expose:

```
POST /api/chat
```

Request:

```
{
  "message": "I have a headache"
}
```

Response:

```
{
  "response": "..."
}
```

Also create:

```
GET /
```

that confirms the backend is running.

Use proper Pydantic models for request/response validation.

4. Gemini Integration

Create a separate service:

```
backend/services/gemini_service.py
```

Do not put Gemini API logic directly inside `main.py`.

Load the API key from:

```
GEMINI_API_KEY
```

using environment variables.

Never hardcode the API key.

Use the current official Gemini Python SDK/API approach and keep the Gemini model name configurable through an environment variable such as:

```
GEMINI_MODEL=...
```

The application should return a useful error message if the API key is missing or the Gemini API request fails.

5. Healthcare System Prompt

Create:

```
backend/prompts/system_prompt.py
```

The system prompt should define the AI as a **healthcare information and guidance assistant**, not a doctor.

The AI should:

- Provide general health information.
- Explain symptoms in simple language.
- Ask relevant follow-up questions.
- Encourage users to consult qualified healthcare professionals when appropriate.
- Clearly identify uncertainty.
- Use calm, understandable language.

The AI must NOT:

- Claim to diagnose a disease.
- Present a diagnosis as certain.
- Pretend to be a doctor.
- Prescribe medication or dosage.

- Tell users to ignore serious symptoms.
- Replace professional medical care.

Include an appropriate disclaimer in the chatbot's behavior.

For emergency situations, the AI should encourage the user to seek appropriate urgent medical care rather than attempting to handle the emergency itself.

6. Frontend

Create a modern healthcare chatbot interface.

The page should contain:

Header

 HealthAI
AI Healthcare Guidance Assistant

Chat area

Display:

- User messages on the right.
- AI messages on the left.
- Different styling for user and AI messages.
- Scroll automatically to the newest message.

Input area

Include:

[Type your health question...] [Send]

Allow the user to press **Enter** to send a message.

Prevent sending empty messages.

7. User Experience

Add:

- Loading/typing indicator while waiting for Gemini.
- Disable the send button while a request is processing.
- Clear error messages if the backend or Gemini API fails.
- Responsive design for desktop and mobile.

- A clean, professional healthcare-inspired interface.

Do not overcomplicate the design.

The UI should look like a real college software project rather than a basic HTML demo.

8. API Communication

The frontend should use JavaScript `fetch()` to call:

```
POST /api/chat
```

Example flow:

```
User enters message
↓
JavaScript
↓
POST /api/chat
↓
FastAPI
↓
Gemini Service
↓
Gemini API
↓
FastAPI
↓
JSON response
↓
JavaScript
↓
Display AI response
```

9. Security

Implement basic security practices:

- Never expose `GEMINI_API_KEY` to frontend JavaScript.
- Keep `.env` out of Git.
- Add `.env` to `.gitignore`.
- Provide `.env.example`.
- Validate incoming requests.
- Handle API exceptions safely.
- Do not expose API keys or internal stack traces in API responses.

10. requirements.txt

Include all packages actually required by the implementation.

For example, depending on the Gemini SDK being used:

```
fastapi
uvicorn
python-dotenv
pydantic
google-genai
```

Do not add unnecessary dependencies.

11. README

Create a useful README containing:

Project title

HealthAI — AI Healthcare Guidance Assistant

Description

Explain that this is the first milestone of a larger college project.

Current features

- AI healthcare chatbot
- FastAPI backend
- Gemini integration
- Healthcare-focused system prompt
- Responsive chat UI

Planned features

Mention that future milestones will add:

- Context-aware conversation
- Symptom extraction
- ML-based risk classification
- Emergency symptom detection
- General health guidance
- Conversation history
- Optional health profile
- Authentication
- Database

- Deployment

Installation

Explain:

```
git clone <repository-url>
cd HealthAI
pip install -r requirements.txt
```

Create `.env`:

```
GEMINI_API_KEY=your_api_key_here
GEMINI_MODEL=your_model_name
```

Running the application

Provide the exact command needed to start FastAPI with Uvicorn.

Explain how to open the frontend/application in a browser.

12. Coding Standards

Follow these principles:

- Clean and readable Python.
 - Modular architecture.
 - Meaningful variable/function names.
 - Type hints where appropriate.
 - Comments only where useful.
 - No unnecessary complexity.
 - No hardcoded secrets.
 - No unnecessary libraries.
 - Keep frontend, API routes, AI service, and prompts separated.
-

13. Important Scope Restriction

This is **Milestone 1 only**.

Do NOT implement:

- MongoDB
- Authentication
- JWT
- User accounts

- Conversation persistence
- ML model
- Symptom extraction
- Emergency classification system
- Health profiles
- Deployment configuration

However, structure the code so these can be added later without rewriting the entire application.

14. Final Deliverable

After implementing the project:

1. Show the complete folder structure.
2. Provide every required source file.
3. Explain what each file does.
4. Provide installation commands.
5. Provide the `.env` configuration.
6. Provide the command to run the application.
7. Explain how the frontend communicates with FastAPI.
8. Explain how FastAPI communicates with Gemini.
9. Explain how the architecture can be extended in Milestone 2.
10. Check the code for obvious errors before presenting it.

The final result should be a **fully working HealthAI v0.1 web chatbot**, not pseudocode or a partial implementation.